

DUPLEXPILOT: 面向全双工语音计划内冷恢复的请求归属状态虚拟化

Anonymous authors

Paper under double-blind review

ABSTRACT

全双工语音服务的执行状态相互关联，并分布于语言生成、token 缓冲与声学合成等多个阶段。因此，在挂起期间回收会话私有资源时，仅恢复语言模型的键值缓存并不足够：待完成的计算还必须与新接收的输入及已经交付的音频保持一致。本文提出 DUPLEXPILOT，一种基于请求归属状态虚拟化、支持一致冷恢复的语音服务运行时。DuplexPilot 将逻辑会话状态与物理执行资源分离，在一致切面上捕获模型、桥接与声学状态。其恢复协议保留输入与交付记录，建立独立的主机内存检查点，并在恢复后的状态与执行绑定通过验证后，才允许继续计算和提交输出。共享模型权重保持常驻，无需重新训练或改变采样语义。有界打包检查点传输与完整性校验模块进一步降低恢复开销。受控实验表明，在所评估配置下，系统能够保持 token 与波形续算结果的一致性。相比逐轮完整历史重建，DuplexPilot 将从 Worklet 恢复到首个新计算 PCM 帧渲染的区间缩短了 12.5 秒，该数值为配对减少量的中位数。在另一组匹配的传输微基准中，优化的传输与校验模块相较同步实现，将检查点传输开始到接收端就绪的区间降低了 16%，该数值为逐对相对降幅的中位数；该实验没有测量 PCM 渲染或端到端恢复时延。上述结果为可恢复全双工语音服务提供了显式执行状态基础，并刻画了跨阶段会话恢复中的资源-时延权衡。

1 引言

全双工语音交互支持同时听说，包括重叠语音、打断和附和 (Nguyen et al., 2023; Veluri et al., 2024)。Moshi 与 Lychee-FD 等模型协调语言和声学多路流 (Défossez et al., 2024; Liu et al., 2026)，因此服务运行时必须保持模型进度、声学消费和音频交付的一致性。

本文研究计划内冷恢复：在明确挂起时释放会话私有 GPU 状态，并在权重保持加载时从主机状态继续执行。热驻留避免重建，完整重建重新构造被释放状态，部分卸载则保留选定阶段；与普通对话停顿不同，该操作会停止执行并在稍后恢复。本文统一使用以下术语：计划内冷恢复 (planned cold resumption) 指明确暂停、释放请求私有状态并随后恢复；*Save* 指从暂停接受到独立检查点就绪，*Ready* 指从恢复接受到执行就绪，

Gap 指缓存 PCM 耗尽到首个新计算 PCM 之间的区间。*first-new-PCM* 指真正新计算 PCM 的第一帧被 AudioContext 渲染；缓存或重放音频不计入。外部账本 (external ledger) 记录计算回退不改变的当前事实，包括已接受输入、取消和交付 / 渲染记录。续算保真性 (continuation fidelity) 只表示 token 与 PCM 的数值连续性，不表示语义等价或听感等价。本文用检查点传输表示检查点字节搬运，用 *request ID* 表示逻辑身份，并统一写作 *Flow / HiFT*。PagedAttention、SGLang、CachedAttention、InferCept 和 Llmunix 提供 KV 复用、拦截、卸载或请求迁移 (Kwon et al., 2023; Zheng et al., 2024; Gao et al., 2024; Abhyankar et al., 2024; Sun et al., 2024); LiveServe、vLLM-Omni 和 Execution-State Capsules 分别提供语音感知调度、重连或计算图边界恢复 (Zhi et al., 2026; vLLM-Omni Contributors, 2026; Su, 2026)。这些机制尚未共同定义覆盖模型状态、桥接内容、声学连续状态及外部输入和交付事实的一致切面。

因此本文提出三个问题：联合状态能否保持新生成 token 和波形后缀；释放私有资源的时延与内存代价是什么；有界传输能否减少等待、又会带来多少暂存开销？本文只研究常驻服务内逻辑请求的计划内挂起，不覆盖任意进程崩溃、客户端断线容灾、语义质量和不受限并发计划内冷恢复。

核心困难是跨阶段一致性：token 可能仍在桥接中，而合成和播放尚未完成；物理执行、采样选择、KV 块和声学上下文可以变化，但逻辑身份保持不变，已接受输入和已交付音频不能回退。应用一致快照和恢复原则 (Chandy and Lamport, 1985; Elnozahy et al., 2002) 需要显式的归属、进度和提交规则；图 1 概括这些依赖及声学执行粒度。

本文提出 DUPLEXPILOT，一种请求归属状态虚拟化运行时：在一致切面捕获模型状态、桥接待处理内容、声学连续状态和阶段进度，并将输入接收与交付记录置于计算回退之外。系统在释放私有资源前建立独立主机检查点，恢复时重建合法绑定并在执行或输出提交前联合验证；有界打包与完整性校验降低搬运开销而不改变恢复合同或采样语义。

同模型受控实验比较热驻留、部分卸载、联合卸载和逐轮重建。在所测试条件下，DuplexPilot 保持 token 和波形续算；相对重建参考，从 Worklet 恢复到首个新计算 PCM 帧渲染的区间配对减少中位数约 12.5 秒；独立的传输微基准中，检查点传输到接收端就绪区间的配对相对降幅中位数为 16%，该实验不测量渲染。本文贡献包括：

- 跨物理资源变化关联模型执行、桥接内容和声学连续状态的请求归属表示。
- 在保留外部输入与交付事实的同时，协调捕获、资源交还、恢复和验证后提交的联合冷恢复协议。
- 在固定恢复合同下对续算保真性、驻留权衡和传输效率的评价。

2 相关工作

全双工语音模型与评价。Duplex Conversation 协调对话状态检测、附和与插话处理 (Lin et al., 2022)；后续双工模型采用时间切片实现听说并行 (Zhang et al., 2024)。dGSLM 和 SyncLLM 分别研究双通道语音建模与真实时间同步 (Nguyen et al., 2023; Veluri

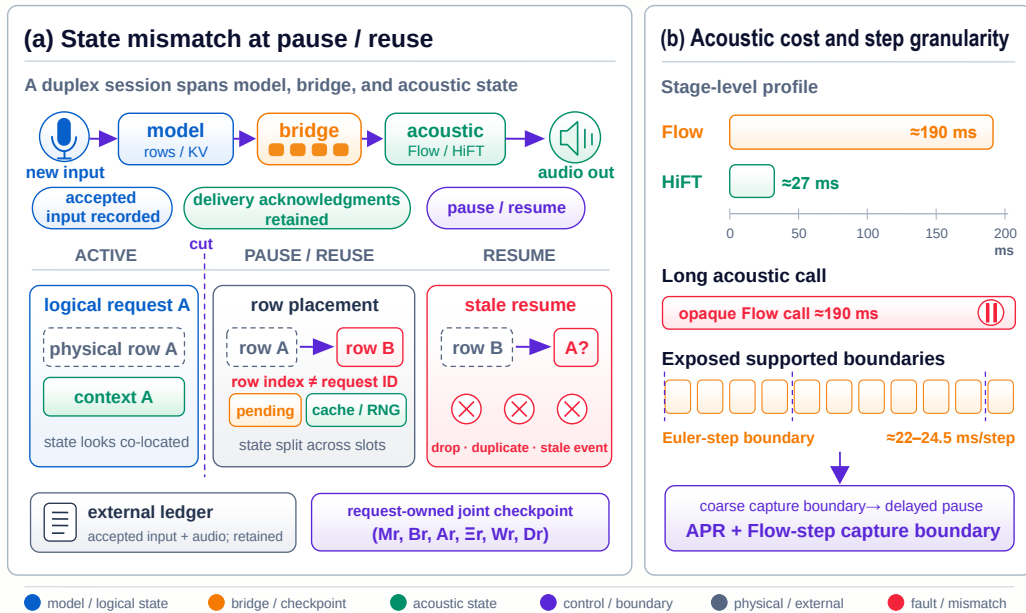


图 1: 显式状态虚拟化的研究动机。(a) 将逻辑状态绑定到临时执行位置, 可能在暂停与复用导致身份失效、桥接内容遗漏或过期输出事件。(b) 在所分析的执行栈中, Flow 与 HiFT 耗时分别约为 190 ms 和 27 ms; 受支持的 22–24.5 ms Euler 步边界提供了更细的捕获与恢复粒度。这些是阶段级观察而非端到端加速, 步级分块为示意。

et al., 2024)。Moshi 结合语音生成与文本规划 (Défossez et al., 2024); SpeechGPT 和 PSLM 分别探索跨模态及文本–语音并行生成 (Zhang et al., 2023; Mitsui et al., 2024); LLaMA-Omni、LLaMA-Omni 2 和 OmniFlatten 进一步面向低时延或端到端语音交互 (Fang et al., 2025a;b; Zhang et al., 2025)。Freeze-Omni 将训练后的语音组件连接到冻结的语言骨干 (Wang et al., 2025), Lychee-FD 则分离声学及语义建模 (Liu et al., 2026)。FD-Bench 评价回应、打断行为及其时序 (Peng et al., 2025); Full-Duplex-Bench 家族包括原始 v1、关注重叠处理的 v1.5 版本 (Lin et al., 2025c;b), 以及进一步考察多轮指令遵循与任务能力的 Full-Duplex-Bench-v2 框架 (Lin et al., 2025a)。近期综述系统梳理了语音语言模型的架构、训练和评价空间 (Arora et al., 2025)。这些研究与 DuplexPilot 的执行状态管理形成互补; 后者无需重新训练底层模型。对话质量与续算保真性仍然属于不同的评价维度。本文研究执行状态恢复, 不改变模型本身或对话能力。

状态化与语音感知服务。Orca、vLLM 和 SGLang 分别提供迭代级调度、分页 KV 分配和共享前缀复用 (Yu et al., 2022; Kwon et al., 2023; Zheng et al., 2024)。Sarathi-Serve 采用分块预填充, DistServe 则分离预填充与解码资源 (Agrawal et al., 2024; Zhong et al., 2024)。CachedAttention 和 Prompt Cache 分别研究跨话轮及提示模块的可复用注意力状态 (Gao et al., 2024; Gim et al., 2024); InferCept 在外部交互期间协调状态保留、卸载与重计算 (Abhyankar et al., 2024); Llumnix 迁移运行中的请求及其状态 (Sun et al., 2024)。面向语音服务, LiveServe 利用播放与语音信号进行调度和 KV 驻

留管理 (Zhi et al., 2026), vLLM-Omni 提供持久会话身份以及带事件重放的重新连接能力 (vLLM-Omni Contributors, 2026)。DuplexPilot 通过协调会话私有资源交还后的模型、桥接与声学状态恢复, 补充这些调度与驻留机制, 关注的是跨模型、桥接和声学阶段的联合逻辑切面。

检查点与恢复语义。一致快照与回滚恢复协调局部状态、在途信息及外部可见效果 (Chandy and Lamport, 1985; Elnozahy et al., 2002)。MillWheel 在其容错边界内, 将持久状态更新与输出检查点、重复抑制相结合 (Akidau et al., 2013)。PhoenixOS 使 GPU 检查点保存和恢复与应用执行重叠 (Wei et al., 2025), GhostServe 通过主机驻留的编码冗余保护 KV 状态, 以支持设备故障恢复 (Jayakody et al., 2026)。Execution-State Capsules 显式表示计算图边界上 KV 之外的连续执行状态 (Su, 2026)。DuplexPilot 将这些原则应用于计划内会话挂起: 一致恢复模型、桥接与声学进度, 同时保留约束输出提交的输入接收记录与交付记录。其恢复单元是常驻服务内部的逻辑请求, 而非整个 GPU 进程, 因此恢复粒度不同于进程级检查点和设备故障恢复。

3 请求归属状态虚拟化

DuplexPilot 虚拟化执行状态而不改变原生监听以及文本、语音和控制生成 (Liu et al., 2026)。图 2 保留四个运行时组件: M1 请求归属、M2 桥接与声学推进、M3 联合恢复、M4 经验证的传输。实现面向常驻服务中单个逻辑请求的计划内挂起, 不是进程崩溃恢复或通用调度器。其约束是合法跨阶段切面、独立于物理位置的逻辑归属、单调外部事实以及验证后执行与发布。

3.1 请求归属状态与模型执行

对于请求 r 在合法切面 c 上的可恢复状态, 定义

$$C_r(c) = (M_r, B_r, A_r, \Xi_r, W_r, D_r), \quad (1)$$

这六个分量是边界互不重叠的请求归属记录: M_r 保存模型 KV / 非 KV 及采样器历史; B_r 是已产生但未消费声学 token 的有序队列; A_r 保存解码器、Flow / HiFT、vocoder 和未提交 PCM 的连续状态; Ξ_r 保存实际消耗的随机状态; W_r 保存阶段与游标; D_r 保存身份、版本、lease、permit 和绑定元数据。token 侧待输出属于 B_r , 波形侧待输出属于 A_r , 已接收输入、取消和已提交 / 渲染区间保留在外部账本 $\mathcal{L}_r$ 中。联合恢复按 $D_r, M_r \rightarrow B_r \rightarrow (A_r, \Xi_r) \rightarrow W_r$ 准备并原子发布; 只有明确声明的部分释放策略可以省略阶段。共享权重、arena 和工作区独立于该契约。

RSV 将持久历史绑定到逻辑请求, DSV 将分歧状态打包到临时执行行并把更新归还所有者。因此物理输出 $[A, B]$ 而采样选择 $[B]$ 时, 必须按 request ID 解析 B , 不能取行零。缺失、重复、过期或跨 generation 身份均被拒绝。合法 KV 块通过分页后端分配 (Kwon et al., 2023), runner 与 scheduler 共享恢复后的活动请求对象; 只复制字段而不恢复绑定, 会丢失调度许可或处理器回写。beam 与并行序列身份不在评估范围。

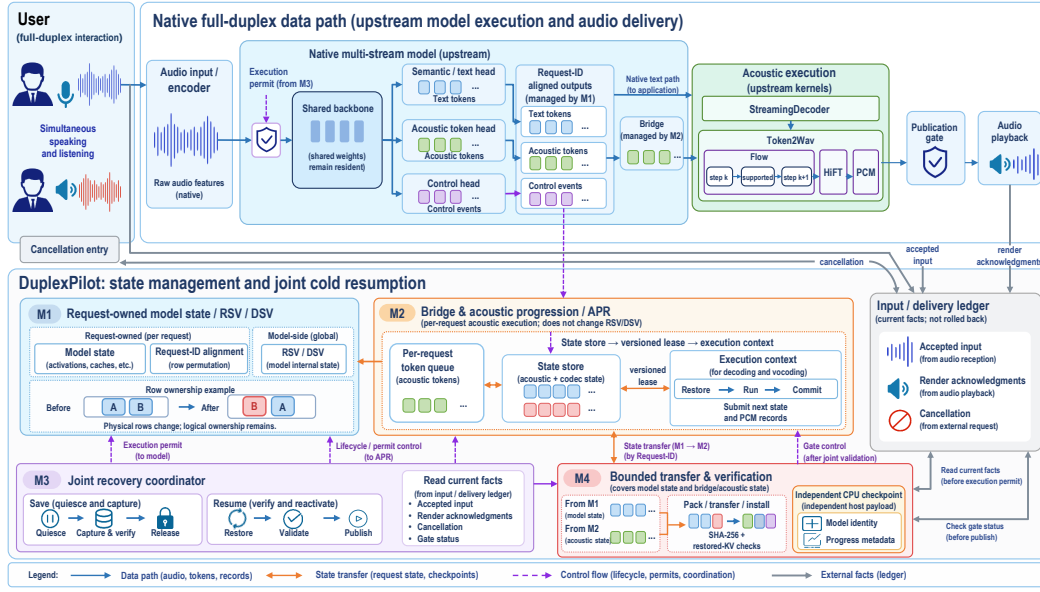


图 2: DuplexPilot 系统架构。原生多流模型与流式声学后端构成数据路径。M1 维护跨执行行的请求归属；M2 管理桥接内容与声学连续状态；M3 协调一致捕获、资源交还和验证后的恢复；M4 传输并校验检查点数据。输入与交付记录在计算回退之外。实线表示数据流，虚线表示运行时控制。

表 1: APR 阶段契约。各阶段消费所列输入，并且只有满足相应提交规则时才能发布结果。

阶段	输入	提交规则
Restore	检查点与当前 lease	建立执行上下文，不发布输出。
Consume	有序声学 token 批次	在同一请求和版本下推进消费位置。
Synthesize	解码器状态、Flow / HiFT 上下文和 RNG	产生 PCM 及下一声学状态。
Commit	下一状态与输出记录	仅在 lease 和 permit 仍有效时提交。

3.2 桥接状态与声学推进

APR 传递带请求、流、generation、序号和版本标识的不可变 token 消息。带版本 lease 恢复 StreamingDecoder 和 Token2Wav 上下文，消费有序批次并连同 PCM 记录提交下一状态；取消或重新分配后，过期 worker 不能提交。

四个 APR 阶段契约汇总于表 1。

已产生但未消费的 token 属于 B_r ；部分解码、Flow / HiFT 和 vocoder 缓冲以及未提交 PCM 属于 A_r ；初始化和 flush 标志属于 W_r ；声学实际使用的随机生成器属于 Ξ_r 。Flow 步边界只暴露进度，不抢占在途 CUDA kernel 或改变模型 batch 形成；重新绑定只要求兼容的声学上下文。

3.3 一致捕获与联合恢复

协调器关闭发放与发布入口，完成待提交转换并建立一致切面 (Chandy and Lamport, 1985)；尚未消费的桥接内容被显式捕获。独立主机检查点完成验证后才交还私有引用和 KV 块，捕获失败则原资源保持权威。图 3 的七阶段为 PAUSE、QUIESCE/CUT、CAPTURE/VERIFY、RELEASE、RESUME、RESTORE/VERIFY 和 PUBLISH；只有 PUBLISH 重新开放执行与输出，输入及交付账本贯穿全过程。

每个检查点、桥接 token、PCM 块和状态提交都携带如下信封：

$$q = (request_id, generation, state_version, cut_id, lease_epoch), \quad e = (q, sequence, producer_epoch).$$

request_id 不可变；generation 在迁移或取消后恢复时递增；每次接受账本变更都递增 state_version；cut_id 唯一标识本次捕获切面；每次 lease 撤销 / 重新发放都递增 lease_epoch。生产者只有在信封仍为当前 q 时才能追加，消费者只有在原子记录接收后才能推进 sequence。因此，仅比较 request 和 generation 并不能排除旧切面或 ABA lease。

协议维护六条不变量：归属（每个可恢复对象只属于一个逻辑请求）；桥接守恒（每个已产生未消费 token 恰好位于一个检查点或队列位置）；账本单调性（旧计算不得覆盖输入、取消和交付事实）；permit 新鲜性（旧 generation、版本、lease 或 worker 不能提交）；发布安全（联合验证前禁止执行和输出）；PCM 有序性（同一 request/generation 下 sequence 严格递增且不重复）。RESUME 时检查身份、生命周期、物理映射、参与状态、载荷、绑定和全部信封字段，恢复保存的消费位置但不隐藏随后接收的输入。完整流程见附录 B 的算法 1；正文只保留两个线性化点和发布守卫。

定义 $\text{JointValid}_r(M, B, A, q)$ 包含 $\bigwedge_{s \in \{M, B, A\}} \text{Valid}_{r,s}$ ，并与信封一致、归属、桥接守恒、账本水位、permit 新鲜性和 PCM 前缀有效性取合取。发布满足

$$\begin{aligned} \text{Publish}_r \iff \exists q : \text{JointValid}_r(M, B, A, q) \\ \wedge \text{LedgerPreserved}_r \wedge \text{PermitCurrent}_r \wedge \text{PCMOrdered}_r. \end{aligned} \quad (2)$$

反向蕴含是实现规则：只有 PUBLISH 能重新开放执行，且必须在原子检查守卫后执行。相同 q 保证 (M, B, A) 来自同一切面并排除 ABA；单调账本拒绝回滚，producer_epoch 拒绝迟到 worker，序列前缀检查阻止 PCM 重复或乱序。任一检查失败都在发布前中止；续算相等还要求匹配的后续输入 / 控制和可重复数值执行 (PyTorch Contributors, 2024)。

3.4 有界传输与完整性校验

状态适配器独立指定字节范围，M4 使用 pinned 主机暂存、设备工作区和完成事件执行有界 gather-transfer-scatter (Moens, 2024)；传输和 SHA-256 校验完成前，缓冲区

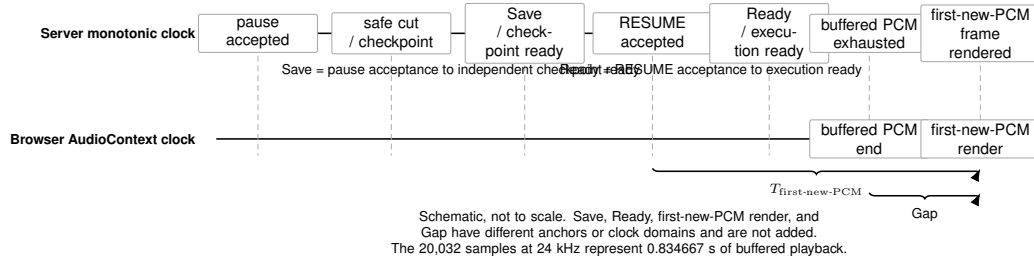


图 3: 联合挂起 / 恢复协议及测量端点。图示不按时间比例绘制，分别显示服务端和浏览器时钟线：Save 和 Ready 属于服务端区间，首个新 PCM 渲染和 Gap 使用浏览器 AudioContext；这些区间具有不同锚点，不能相加。

不能复用，恢复后的 KV 还需在新映射中检查。这些检查确认字节完整性，式 (2) 决定能否发布；payload、pinned 暂存、进程 RSS 和 GPU 分配属于不同统计口径。优化与同步路径保持相同检查点内容、恢复义务、模型计算和采样语义，打包只改变搬运成本。共享权重、执行 arena 和传输工作区单独计量，部分卸载与联合卸载采用同一续算合同。

4 实验

4.1 实验环境

为全面评估所提出的全双工服务运行时在功能正确性、恢复效率和交互服务能力方面的表现，本文在固定的硬件和执行配置下开展实验。所有实验均使用两张 A100-SXM4-40GB GPU，其中模型执行和声学执行分别部署，共享模型权重保持常驻。实验主要从三个方面展开分析：冷恢复，考察释放请求私有状态后的恢复连续性及时延-资源权衡；传输与校验，隔离固定恢复合同下的检查点传输与校验开销；交互与外部协议，考察不同全双工事件下的交互表现以及不同协议所带来的端点差异。各实验的具体数据组织、对比策略、评价指标和统计分母将在对应小节中分别说明。

4.2 冷恢复：正确性及时延-资源权衡

冷恢复实验在两张 A100-SXM4-40GB GPU 上完成，负载采用冻结的公开 HumDialogBench 测试归档中的语句级样本 (Wang et al., 2026; ASLP-lab, 2026)。该实验共统计 96 条运行记录。我们比较 Hot、Rebuild、Joint 和 Hybrid 四种状态恢复策略，并重点考察恢复连续性、首个新计算 PCM 渲染区间、post-buffer Gap 以及状态恢复所产生的资源开销。各策略的详细运行端点统计及其来源见附录。

测量端点使用两条独立时钟线。Save、RESUME 接受和 Ready 使用服务端单调时钟；Worklet 恢复、缓存 PCM 耗尽和首个新计算 PCM 渲染使用浏览器 AudioContext 时钟。主指标定义为

$$T_{\text{first-new-PCM}} = t_{\text{new}} - t_{\text{W, resume}}. \quad (3)$$

Save 是从暂停接受到独立检查点就绪的区间，Ready 是从 RESUME 接受到执行就绪的区间，Gap 是从缓存 PCM 耗尽到首个新计算 PCM 的区间。这些区间具有不同锚点，不能相加构成端到端时延；12.5 秒来自逐运行配对的主指标差值，而不是描述性表格中位数之差。

表 2: 冷恢复的主要连续性、时延和资源结果。表中保留技术连续性结果、首个新 PCM 渲染端点、相对 Rebuild 的配对减少量以及独立的 E2 资源口径；数值单位为秒或 MiB。valid/scheduled 保留全部计划运行；详细 Save、Ready、Gap、失败标签和来源记录见附录。

指标	Hot	Rebuild	Joint	Hybrid
状态/恢复边界	保留活动上下文； 时延参考	释放私有状态； 顺序重建	释放私有状态； 联合检查点	释放模型/KV； 保留声学状态
连续性通过：有效/计划	24/24；	23/24；	24/24；	21/24；
E1 首个新 PCM 渲染 (s)	1.184	14.451	1.899	1.349
E2 首个新 PCM 渲染 (s)	1.168	14.400	1.872	1.315
			E1: 12.547 (10/11)	E1: 12.517 (8/11)
相对 Rebuild 的配对减少量 (s)	N/A	0.000	E2: 12.507 (13/13; 12 组)	E2: 13.000 (13/13; 12 组)
E2 检查点载荷 (MiB)	N/A	NM	672.207	22.416
E2 受管理 pinned 缓冲 (MiB)	N/A	NM	256	128
E2 CPU 持有 RSS (MiB)	10534.685	10551.084	11477.773	10699.230
E2 声学 GPU 净回收 (MiB)	0.000	649.702	585.702	0.000
E2 模型 GPU 净分配变化 (MiB)	0.000	-3.511	+60.489	+60.489
计划运行：96 条；技术通过：92 条；每策略分母：E1 11 + E2 13 = 24				

注：连续性通过要求当前 request/generation/permit 有效，模型、bridge、声学状态及 progress/RNG 校验通过，post-cut token/PCM 后缀完整，输入/取消/交付账单单调，无过期或重复输出，并且未来输入/control 和数值设置匹配。四种策略共享模型、切面、resume 信号、播放器和连续性合同；Hybrid 只改变驻留边界，因此与 Joint 的比较是匹配条件下的时延-资源权衡。配对单元格报告首个新 PCM 渲染的正向减少量；E2 包含 13 个计划配对中的 12 个来源组，E1 报告可评价配对数。N/A 表示该策略不触发相应事件，NM 表示冻结清单中没有对应测量；二者都不表示零。不同资源口径彼此独立，不能直接相加。端点定义和 Gap 统计见附录。

表 2 汇总了四种状态恢复策略的连续性、主要时延和资源开销。矩阵包含 96 条计划运行和 92 条技术通过记录；每个策略的分母为 24 条 (E1 为 11 条，E2 为 13 条)。表中的通过是技术连续性合同，不等同于语义回答质量。Joint 恢复的状态包括模型 KV/non-KV 状态、待处理 bridge 内容、StreamingDecoder 和 Token2Wav 连续状态、Flow / HiFT 缓存、实际随机上下文、stage progress 以及请求身份和绑定描述符。已接受输入、取消和交付事实保存在独立的单调账本中，不被旧检查点覆盖。

Hot 保留活动上下文，因此只是时延参考，不满足冷释放。Rebuild 释放请求私有状态，并执行本文作为严格参考的顺序完整历史重建。Hybrid 释放模型/KV 状态，但保留必要声学状态，是部分释放策略，而不是第二个 Joint 实现。四种策略仍共享同一模型、切面、输入、resume 信号、未来 input/control、播放器、校验合同和计划矩阵，因此 Joint 与 Hybrid 的比较是在匹配条件下比较明确驻留边界带来的时延-资源权衡。

配对减少量来自匹配记录。Joint 在 E1 的 11 个计划配对中有 10 个可评价，首个新 PCM 渲染区间减少 12.547s；E2 为 13/13 个可评价配对、12 个来源组，减少 12.507s。Hybrid 对应为 E1 的 8/11 个配对、减少 12.517s，以及 E2 的 13/13 个可评价配对、12

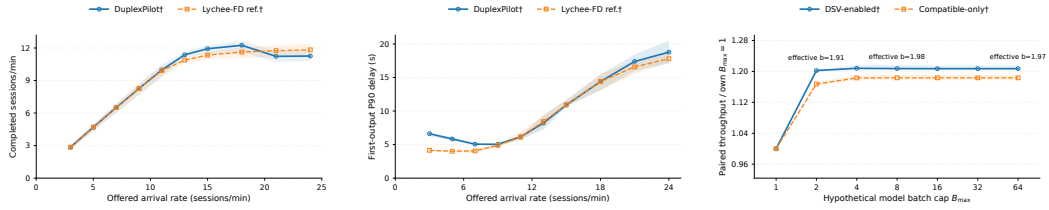


图 4: 校准后的端到端吞吐量随负载变化。(a) 原生 Lychee-FD 与 DuplexPilot 的成对场景; 候选/基线比值锚定于已测 APR/Affinity 运行。(b) 同一成对场景中首个输出 P90 随到达负载的变化; 曲线为 5 次重复运行的均值, 阴影表示四分位距。(c) Lychee-FD 服务栈上随最大模型 batch $B_{\max}$ 变化的吞吐扩展性。

个来源组, 减少 13.000s。这些数值不是两个未配对总体中位数相减得到的; 附录报告对应的 Gap 中位数, 并采用同 generation 缓存耗尽后的定义。

资源数据说明了这一权衡的成本和统计口径。E2 中 Joint 和 Hybrid 的独立检查点载荷为 672.207 和 22.416 MiB, 受管理 pinned 缓冲为 256 和 128 MiB, CPU 持有 RSS 为 11477.773 和 10699.230 MiB。Rebuild 在冻结清单中没有对应检查点载荷事件, 因此标记为 NM 而不是零; Hot 保留活动状态, 因此标记为 N/A。模型 GPU 净分配变化为正表示净增加, 为负表示净回收; 各资源口径彼此独立, 不能直接相加。

4.3 吞吐量与扩展性

我们首先分析基于实测比值校准的 DuplexPilot 负载相关吞吐场景, 该场景运行在原始 Lychee-FD 服务栈上。图 4(a) 显示, 在 3–7 sessions/min 的低负载区间, 吞吐量主要跟随到达率, 候选由于启动路径略低于原生系统。进入容量受限区间后, 两条曲线开始分离: 在 13 sessions/min 时, 校准场景中 DuplexPilot 为 11.38 sessions/min, Lychee-FD 为 10.88; 在 15 sessions/min 时为 11.93 对 11.34; 在 18 sessions/min 时为 12.25 对 11.62。在最高负载点, 测得的 C/N16 锚点反转了排序: DuplexPilot 为 11.26, 而 Lychee-FD 为 11.83。因而, 成对比值同时包含增益和回退, 不能概括为普遍吞吐优势。

图 4(b) 展示了这些吞吐量背后的时延权衡。在最低负载点, DuplexPilot 的首个输出 P90 较高 (6.63 s 对 4.14 s); 在中负载区间时延略低 (13 sessions/min 时为 8.23 s 对 8.44 s), 而在最高负载点 P90 又更高 (24 sessions/min 时为 18.77 s 对 17.80 s)。四分位阴影展示了重复运行之间的波动, 同时避免把单次合成运行误当成独立工作点。这样可以把吞吐量变化与端点响应速度结合起来解释。

最后, 图 4(c) 展示了同一服务栈中模型 batch 上限的影响。相对于各自的 $B_{\max} = 1$, DuplexPilot 在 $B_{\max} = 2$ 时达到约 1.20, 并在约 1.21 附近饱和; Lychee-FD 达到约 1.18。两者差距较小且较早饱和, 与已测成对比值一致: 提高上限能够恢复部分混批机会, 但不能保证端到端吞吐单调增加。

4.4 传输与校验效率

为进一步分析检查点传输路径的效率，我们在相同的负载实例和恢复合同下比较 Sync 与 Optimized 两种实现。两种实现使用相同的状态载荷、输入来源、重复过程、接收端校验和接受规则。Sync 表示同步完成检查点传输和接收确认的路径，Optimized 表示采用打包传输、传输与校验重叠的优化路径。因此，该实验只改变检查点传输与校验的实现方式，不改变恢复语义和后续执行过程。我们主要统计状态从发送端开始传输到接收端达到就绪状态之间的时间，并同时记录暂停 RSS 和采样峰值 RSS 的变化。

在所有归档的匹配运行实例中，Optimized 的传输至就绪时间均早于 Sync。Optimized 相对于 Sync 的绝对减少量中位数为 0.533s，逐实例相对减少量的中位数为 16.00%。其中，逐实例相对减少量定义为

$$r_i = \frac{T_i^{\text{Sync}} - T_i^{\text{Optimized}}}{T_i^{\text{Sync}}}, \quad T_i = t_{i,\text{receiver ready}} - t_{i,\text{state transmission start}}.$$

采用逐实例相对减少量而不是两个配置整体中位数之比，可以避免不同负载实例之间的时延差异改变总体结果的解释。所有匹配实例均获得正向收益，说明该优化并非只在少数特殊负载上有效，而是能够稳定地减少检查点传输路径中的等待时间。

传输效率的提升伴随着额外的内存开销。与 Sync 相比，Optimized 的暂停 RSS 和采样峰值 RSS 配对增加中位数分别为 243.2 和 182.2 MiB。这一开销主要来自状态打包、暂存和接收端校验过程中需要保留的中间数据。因此，Optimized 体现的是传输等待时间与 CPU 内存占用之间的工程折中：在恢复合同保持不变的情况下，打包和重叠校验能够缩短检查点传输时间，但需要额外的主机内存支持。16.00% 是逐实例相对减少量的中位数，不能解释为配置中位数之比，也不能作为额外倍率直接叠加到冷恢复实验中的时延减少量上。该批次没有测量 Worklet 恢复、首个新 PCM 渲染、播放结束或端到端恢复时延。

4.5 交互与外部协议

为考察运行时在全双工交互场景中的状态转移和输出行为，我们采用公开 HumDialogBench 测试归档整理的受控交互负载，并使用 FD-Bench 风格的事件定义 (ASLP-lab, 2026; Peng et al., 2025)，覆盖普通回复、用户插话、反向通道和停顿等事件类型。内部路径使用固定的事件定义、输入节奏和统计分母，并按照 SRR、SIR、SRIR、EIR、FSED 和 IRD 等指标进行评估。与此同时，我们记录 vLLM-Omni 原生热重附着和 LiveServe 风格预加载路径的端点行为，用于分析不同协议下 ready、ACK 和首个 PCM 输出之间的关系。外部路径的模型、输入调度和计时端点与内部受控实验不同，因此只作为协议参照，不作为匹配基线。

本先导中，SRR 定义为符合条件的非打断问句中成功回复的数量除以符合条件的非打断问句总数；SIR 为实际打断机会中检测到正向打断的数量除以实际打断机会总数；SRIR 为实际打断机会中成功完成打断响应的数量除以实际打断机会总数；EIR 为有效事件窗口中助手过早进入的数量除以有效事件窗口总数。FSED 是用户语音结束到匹

配回复开始播放的带符号时延, IRD 是用户插话开始到助手上一段语音结束的带符号时延。因此下文每个比例都显式给出分子和分母。每条路径的六个 source group 作为 source-level bootstrap 重采样单位, 共进行 200,000 次百分位数 bootstrap; 时延只使用有效事件观测, 不对缺失值插补。

表 3 汇总了内部受控路径和外部协议参照的关键测量口径。最终 manifest 包含 6 个已完成 source group、10 个有效问句、3 个实际打断机会, 且没有无效或取消的运行。受控路径的 SRR 为 $7/7 = 100.0\%$ (95% bootstrap CI 为 100.0–100.0), SIR 为 $3/3 = 100.0\%$ ([100.0,100.0]), SRIR 为 $3/3 = 100.0\%$ ([100.0,100.0]), EIR 为 $4/10 = 40.0\%$ ([25.0,50.0]), FSED 中位数为 4445.9 ms ($n = 10$, [3714.6,5582.8]), IRD 中位数为 278.2 ms ($n = 3$, [266.6,898.9])。没有无效 / 取消运行或不可评价 source group; 无效 / 取消事件行同样为 0, 评分器排除了 2 条非指标反向通道记录, reply timeout 为 0。逐 source 重复结果见表 6 及机器可读文件 data/final/interaction_metrics_per_repeat.csv, bootstrap 细节见 data/final/interaction_metrics_bootstrap.csv。原生 Omni 参照的 SRR 为 $6/7 = 85.7\%$ (bootstrap CI 为 [66.7,100.0]%), SIR 为 3/3, SRIR 为 3/3, EIR 为 $9/10 = 90.0\%$ ([70.0,100.0]%), FSED 中位数为 635.5 ms ($n = 9$, [58.5,1324.7]), IRD 中位数为 2068.3 ms ($n = 3$, [564.1,3444.6])。这是现有的原生交互参照, 不是 naive-cold 或同模型匹配基线。对于 vLLM-Omni 原生热重附着, 记录中的请求至 ACK 时间为 1.512 和 1.421 ms, 首 PCM 接收时间为 725.412 和 776.220 ms; PCM 序号没有出现缺失或重复, 但现有记录无法确定首个返回 PCM 所处的具体产生阶段。LiveServe 式组件提前约 9.9s 准备 20 和 15 MiB 的 KV 状态, 使 ready 时间分别提前约 56.65 和 100.01 ms; 然而以共同源窗口为计时锚点时, 首个新 PCM 的接收时间分别晚 23.7 和 36.4 ms。

相对于原生 Omni 交互参照, Lychee/DuplexPilot 减去 Omni 的原始描述性差异为: SRR 增加 14.3 个百分点, SIR 为 0, SRIR 为 0, EIR 减少 50.0 个百分点, FSED 增加 3810.4 ms, IRD 减少 1790.1 ms。这些不是因果基线估计, 因为两条路径的模型、输入调度和计时端点不同; 本批次没有记录 naive-cold 交互基线。Lychee/DuplexPilot 固定采用 400 ms 块末端输入交付; 没有配对的 200 ms 或零量化对照, 因此无法估计 400 ms 的因果影响。该调度最多引入一个 400 ms 的边界量化 (若输入均匀到达, 期望约 200 ms), 所以报告的 FSED 和 IRD 包含这一协议影响。

这些结果表明, ready 时间提前并不必然意味着用户能够更早获得可听输出。Lychee/DuplexPilot 采用 400 ms 块末端输入, 而 Omni 客户端采用 200 ms 先发送再等待的输入协议; 不同协议中的 ready、ACK、首 PCM 接收和 PCM 产生分别对应交互流水线中的不同阶段。因此, 外部协议观察只能用于说明端点关系和协议边界, 不能与内部受控结果合并为统一的冷恢复性能排名。

表 3: 受控交互路径和外部协议参照的测量口径与观察结果。外部协议仅作为观察性参照，不作为匹配基线。

路径或协议	数据与输入方式	主要端点或指标	观察结果与可比性说明
Lychee/DuplexPilot 受控路径	HumDial-FDBench 公开测试音频与 FD-Bench 风格事件定义；普通回复、用户插话、反向通道和停顿；400 ms 块末端输入	SRR、SIR、SRIR、EIR、FSED 和 IRD；SRR 7/7、SIR 3/3、SRIR 3/3、EIR 4/10；FSED 4445.9 ms ($n = 10$)、IRD 278.2 ms ($n = 3$)。bootstrap 95% CI: EIR [25.0,50.0]%, FSED [3714.6,5582.8] ms, IRD [266.6,898.9] ms。6 次重复；无效 / 取消 0/6, reply timeout 为 0	内部受控结果。事件定义、输入节奏和统计分母固定；统计 10 个有效问句和 3 个打断机会。固定 400 ms 交付调度已包含在时延数值中，没有配对 200 ms 消融
vLLM-Omni 原生热重附着	200 ms 先发送再等待；原生 Omni 输入协议	请求至 ACK: 1.512 / 1.421 ms；首 PCM 接收: 725.412 / 776.220 ms	PCM 序号没有缺失或重复；首个返回 PCM 的具体产生阶段未知。模型、输入调度和端点定义不同，不作为匹配基线
LiveServe-style 预加载路径	提前准备 KV 状态；20 / 15 MiB；约 9.9 s 提前执行	ready 时间提前: 56.65 / 100.01 ms；共同源窗口锚点下首个新 PCM 接收: 晚 23.7 / 36.4 ms	更早 ready 不等于更早可听输出。该路径与内部实验的模型、调度和端点定义不同，只用于说明协议层次之间的时间关系

5 讨论与总结

5.1 局限性分析

本文方案的首要适用前提是音频生成侧能够提供可拆分、可导出和可恢复的内部状态。只有当声学后端能够显式暴露必要的缓存、执行进度、待处理输出以及相关运行时上下文时，DuplexPilot 才能完成状态保存、传输、重新绑定和继续执行。如果音频生成过程被封装为无法中断、无法导出和无法恢复的黑盒操作，则该方案无法直接适用。因此，DuplexPilot 并不是面向任意音频生成后端的通用外挂优化，其有效性依赖于声学状态的可观测性和可恢复性。

本文将 DuplexPilot 迁移到 vLLM-Omni 后的测量结果，也不能代表 vLLM-Omni 原生系统能够达到的最优性能。相关结果受到当前迁移实现、输入节奏、调度配置、模型执行路径以及端点定义的共同影响。由于不同系统在模型实现、KV 管理、声学执行方式和输出协议方面存在差异，本文中的 vLLM-Omni 结果只能作为协议和端点层面的观察，不能作为对 vLLM-Omni 整体性能上限的判断，也不能与内部受控结果直接构成统一的系统排名。

此外，当前实验主要覆盖正常的状态保存、传输、恢复和继续执行过程，对故障场景的覆盖仍不充分。检查点传输中断、部分恢复、设备或 worker 失效、checkpoint 损坏、状态版本不兼容以及恢复过程中的回滚和原子提交等问题，尚未得到系统验证。因此，当前结果主要说明运行时在正常恢复路径下的状态管理能力，面向长期运行服务的故障隔离、失败恢复和状态格式演化机制仍需要进一步研究。

5.2 总结

本文的核心工作是将全双工服务中的音频生成状态从隐含的执行上下文转化为运行时可以显式管理的对象。在音频生成状态能够被拆分和恢复的前提下，运行时可以对逻辑请求状态、模型执行状态、桥接状态和声学执行状态进行独立管理，从而支持请求私有资源释放后的状态恢复和全双工连续执行。该方法的价值不在于为某个特定系统提供一个固定的性能倍率，而在于建立了一种面向全双工服务的状态虚拟化抽象。后续工作将重点围绕更通用的声学状态接口、面向故障的恢复机制以及统一公平的跨系统评测协议展开。

AI 使用声明

AI 协助数据分析、图表生成和英文表述修订。

可复现性声明

公开源码仓库位于 <https://github.com/ICALab-nscj/DuplexPilot>。仓库包含核心运行时与检查点合同、公开测试、工作负载生成器、分析工具、配置说明，以及用于构建本文的 LaTeX 源码。生成日志、中间实验结果、归档记录和模型检查点有意不上传。公开数据集以其上游地址、版本和校验和记录；超过 GitHub 普通文件大小限制的数据文件不直接纳入仓库。若第三方检查点因许可不能重新分发，仓库会提供其公开下载地址、精确版本、许可信息和校验和。发布的脚本和论文图表因此暴露可复现的实现接口，但不声称公开私有运行轨迹或受限权重。

REFERENCES

- Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. In-ferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 81–95, 2024. URL <https://proceedings.mlr.press/v235/abhyankar24a.html>.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. MillWheel: Fault-tolerant stream processing at internet scale. *Proceedings of the VLDB Endow-*

ment, 6(11):1033–1044, 2013. URL <https://www.vldb.org/pvldb/vol6/p1033-akida.pdf>.

Siddhant Arora, Kai-Wei Chang, Chung-Ming Chien, Yifan Peng, Haibin Wu, Yossi Adi, Emmanuel Dupoux, Hung yi Lee, Karen Livescu, and Shinji Watanabe. On the landscape of spoken language models: A comprehensive survey. *Transactions on Machine Learning Research*, 2025. URL <https://mlanthology.org/tmlr/2025/arora2025tmlr-landscape/>.

ASLP-lab. HumDial-FDBench: Public dataset for the icassp 2026 HumDial challenge. Hugging Face dataset, 2026. URL <https://huggingface.co/datasets/ASLP-lab/HumDial-FDBench>. Pinned revision ed1770a6265ffe9f0e5cd999290936ccab43482b; dataset card labeled Apache-2.0; accessed 2026-09-26.

K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. doi: 10.1145/214451.214456. URL <https://www.microsoft.com/en-us/research/publication/distributed-snapshots-determining-global-states-distributed-system/>.

Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024. URL <https://arxiv.org/abs/2410.00037>.

E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002. doi: 10.1145/568522.568525. URL <https://research.ibm.com/publications/a-survey-of-rollback-recovery-protocols-in-message-passing-systems>.

Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. LLaMA-Omni: Seamless speech interaction with large language models. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/90d1fc07f46e31387978b88e7e057a31-Abstract-Conference.html.

Qingkai Fang, Yan Zhou, Shoutao Guo, Shaolei Zhang, and Yang Feng. LLaMA-Omni 2: LLM-based real-time spoken chatbot with autoregressive streaming speech synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18617–18629. Association for Computational Linguistics, 2025b. URL <https://aclanthology.org/2025.acl-long.912/>.

Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model

serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference*, pages 111–126. USENIX Association, 2024. URL <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>.

In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

Shakya Jayakody, Youpeng Zhao, Chinmay Dhanraj Nehate, and Jun Wang. Ghost-Serve: A lightweight checkpointing system in the shadow for fault-tolerant LLM serving. In *Proceedings of Machine Learning and Systems*, volume 8, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/d99e8e80a6c41e148db686918dd7eab3-Abstract-Conference.html.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.

Guan-Ting Lin, Shih-Yun Shan Kuan, Jiatong Shi, Kai-Wei Chang, Siddhant Arora, Shinji Watanabe, and Hung yi Lee. Full-duplex-bench-v2: A multi-turn evaluation framework for duplex dialogue systems with an automated examiner. *arXiv preprint arXiv:2510.07838*, 2025a. URL <https://arxiv.org/abs/2510.07838>.

Guan-Ting Lin, Shih-Yun Shan Kuan, Qirui Wang, Jiachen Lian, Tingle Li, Shinji Watanabe, and Hung yi Lee. Full-duplex-bench v1.5: Evaluating overlap handling for full-duplex speech models. *arXiv preprint arXiv:2507.23159*, 2025b. URL <https://arxiv.org/abs/2507.23159>.

Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung yi Lee. Full-duplex-bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. *arXiv preprint arXiv:2503.04721*, 2025c. URL <https://arxiv.org/abs/2503.04721>.

Ting-En Lin, Yuchuan Wu, Fei Huang, Luo Si, Jian Sun, and Yongbin Li. Duplex conversation: Towards human-like interaction in spoken dialogue systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3299–3308, 2022. doi: 10.1145/3534678.3539209. URL <https://arxiv.org/abs/2205.15060>.

-
- Zhenyu Liu, Xuanyu Zhang, Yunxin Li, Qixun Teng, Shenyuan Jiang, Haolan Chen, Mingjun Zhao, Fanbo Meng, Yu Xu, Yancheng He, Baotian Hu, Haizhou Li, and Min Zhang. Hierarchical acoustic-semantic modeling: Modality separation and semantic coherence for full-duplex SLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9264–9280. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.419. URL <https://aclanthology.org/2026.acl-long.419/>.
- Kentaro Mitsui, Koh Mitsuda, Toshiaki Wakatsuki, Yukiya Hono, and Kei Sawada. PSLM: Parallel generation of text and speech with LLMs for low-latency spoken dialogue systems. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 2692–2700. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.151. URL <https://aclanthology.org/2024.findings-emnlp.151/>.
- Vincent Moens. A guide on good usage of `non_blocking` and `pin_memory()` in PyTorch. PyTorch Tutorials, 2024. URL https://docs.pytorch.org/tutorials/intermediate/pinmem_nonblock.html. Created July 31, 2024; updated April 1, 2026; accessed September 25, 2026.
- Tu Anh Nguyen, Eugene Kharitonov, Jade Copet, Yossi Adi, Wei-Ning Hsu, Ali Elkahky, Paden Tomasello, Robin Algayres, Benoît Sagot, Abdelrahman Mohamed, and Emmanuel Dupoux. Generative spoken dialogue language modeling. *Transactions of the Association for Computational Linguistics*, 11:250–266, 2023. doi: 10.1162/tacl_a_00545. URL <https://aclanthology.org/2023.tacl-1.15/>.
- Yizhou Peng, Yi-Wen Chao, Dianwen Ng, Yukun Ma, Chongjia Ni, Bin Ma, and Eng Siong Chng. FD-Bench: A full-duplex benchmarking pipeline designed for full duplex spoken dialogue systems. *arXiv preprint arXiv:2507.19040*, 2025. URL <https://arxiv.org/abs/2507.19040>.
- PyTorch Contributors. Reproducibility. PyTorch 2.9 Documentation, 2024. URL <https://docs.pytorch.org/docs/2.9/notes/randomness.html>. Documentation page last updated November 26, 2024; accessed September 25, 2026.
- Liang Su. Execution-state capsules: Graph-bound execution-state checkpoint and restore for low-latency, small-batch, on-device physical-AI serving. *arXiv preprint arXiv:2606.20537*, 2026. URL <https://arxiv.org/abs/2606.20537>.
- Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 173–191. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/sun-biao>.

-
- Bandhav Veluri, Benjamin N. Peloquin, Bokai Yu, Hongyu Gong, and Shyamnath Gollakota. Beyond turn-based interfaces: Synchronous LLMs as full-duplex dialogue agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21390–21402, 2024. doi: 10.18653/v1/2024.emnlp-main.1192. URL <https://aclanthology.org/2024.emnlp-main.1192/>.
- vLLM-Omni Contributors. Unified full-duplex framework. Official project documentation, 2026. URL <https://docs.vllm.ai/projects/vllm-omni/en/latest/design/full duplex/>. Accessed 2026-09-25.
- Chengyou Wang, Hongfei Xue, Guojian Li, Zhixian Zhao, Shuiyuan Wang, Shuai Wang, Xin Xu, Hui Bu, and Lei Xie. Full-duplex interaction in spoken dialogue systems: A comprehensive study from the ICASSP 2026 HumDial challenge. *arXiv preprint arXiv:2604.21406*, 2026. URL <https://arxiv.org/abs/2604.21406>.
- Xiong Wang, Yangze Li, Chaoyou Fu, Yike Zhang, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma. Freeze-Omni: A smart and low latency speech-to-speech dialogue model with frozen LLM. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63345–63354. PMLR, 2025. URL <https://proceedings.mlr.press/v267/wang25aw.html>.
- Xingda Wei, Zhuobin Huang, Tianle Sun, Yingyi Hao, Rong Chen, Mingcong Han, Jinyu Gu, and Haibo Chen. PhoenixOS: Concurrent OS-level GPU checkpoint and restore with validated speculation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. Association for Computing Machinery, 2025. doi: 10.1145/3731569.3764813. URL <https://doi.org/10.1145/3731569.3764813>.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu. SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15757–15773. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.1055. URL <https://aclanthology.org/2023.findings-emnlp.1055/>.
- Qinglin Zhang, Luyao Cheng, Chong Deng, Qian Chen, Wen Wang, Siqi Zheng, Jiaqing Liu, Hai Yu, Chao-Hong Tan, Zhihao Du, and ShiLiang Zhang. Omniflatten: An

918 end-to-end GPT model for seamless voice conversation. In *Proceedings of the 63rd*
919 *Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*
920 *Papers)*, pages 14570–14580. Association for Computational Linguistics, 2025. doi:
921 10.18653/v1/2025.acl-long.709. URL <https://aclanthology.org/2025.acl-long.709/>.
922
923
924 Xinrong Zhang, Yingfa Chen, Shengding Hu, Xu Han, Zihang Xu, Yuanwei Xu, Weilin
925 Zhao, Zhiyuan Liu, and Maosong Sun. Beyond the turn-based game: Enabling real-
926 time conversations with duplex models. In *Proceedings of the 2024 Conference on*
927 *Empirical Methods in Natural Language Processing*, pages 11543–11557. Association
928 for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.644. URL
929 <https://aclanthology.org/2024.emnlp-main.644/>.
930
931
932 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu,
933 Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and
934 Ying Sheng. SGLang: Efficient execution of structured language model programs. In
935 *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.
936
937
938 Xiangyu Zhi, Peiqi Yin, Sheng Guan, Chenguang Zheng, James Cheng, and Xiao Yan.
939 LiveServe: Interaction-aware serving for real-time omni-modal LLMs. *arXiv preprint*
940 *arXiv:2606.22983*, 2026. URL <https://arxiv.org/abs/2606.22983>.
941
942 Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin
943 Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-
944 optimized large language model serving. In *18th USENIX Symposium on Operating*
945 *Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Associa-
946 tion, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971

A 复现与测量

复现材料。 可复现性声明所述 GitHub 发布包将提供可执行源码、配置与依赖清单、运行命令、图表生成脚本和机器可读结果表，以便重新生成论文中的结果。缺失或不可评价字段将明确报告，不以插值替代。

数据来源与命名。 本文使用公开的 *HumDial-FDBench* 测试归档；正式论文为 Wang 等人 (Wang et al., 2026)，数据页单独引用为 (ASLP-lab, 2026)。官方划分为 Train 8,898、Dev 1,800 和 Test 5,000 条，覆盖九类场景；公开测试归档另含 4,100 个用于评分的 clean 文件。本文冻结的 Hugging Face revision 为 ed1770a6265ffe9f0e5cd999290936ccab43482b，数据卡标注 Apache-2.0。冻结归档中的 WAV 为单声道、16-bit PCM、16 kHz；本文直接使用发布的 WAV 字节和 JSON 标注，不做项目内重采样、降噪、响度归一化或合成混音。样本 ID 保留归档路径 `test/{en,cn}_test_nondev/{scenario}/{base_id}[_add].wav`，clean 文件是独立的评分参照。运行时 cut 从选定完整 WAV 在实验固定的非空状态边界生成，属于项目派生回放切面，不是官方 *HumDial-FDBench* 片段或榜单分数；cut ID、时间戳、来源成员和哈希绑定在运行 manifest 中。该发布物是人工录制的表演语音，不是 TTS；公开论文和数据卡没有说明去标识化或说话人匿名化流程，因此本文不宣称音频已经去标识化。

全文统一使用：*FD-Bench* (Peng 等) 是独立基准，不使用无连字符简称；“*HumDial-FDBench*” 仅作为 *HumDial* 官方数据集标识保留；*Full-Duplex-Bench* 指 Lin 等人的家族，其中 v1 和 v1.5 是离线版本，*Full-Duplex-Bench-v2* 是后续实时框架；*HumDial-FDBench* 是基于 v1.5 协议构建的 *HumDial* 基准，既不是 *FD-Bench* 数据集，也不是 *Full-Duplex-Bench-v2*。本文改编的事件定义统一称为“*FD-Bench* 风格”。

冷恢复实验使用相同的 Lychee-FD 模型与本地 Token2Wav 后端 (Liu et al., 2026)，在两张 A100-SXM4-40GB GPU 上分别部署模型执行和一个声学 lane。共享权重保持常驻，每组比较内部匹配模型、采样、数值、输入和播放配置。复现需要实际执行的源码、依赖与配置清单，包括归档补丁，不能仅依据仓库 commit。

最终矩阵包含 12 个来源组、四种策略和两次重复，共 96 次执行。E1 包含 44 次，E2 包含 52 次，两个环境分别统计。测量批次包含 24 个待处理语音 token，以及 20,032 个 24kHz 缓存 PCM 样本。这些数值描述预定义非空切面，而非自然会话状态分布。名义挂起时间为 30 秒，但时延采用实际事件计量。新音频渲染从 Worklet 恢复开始，计量至同一 *AudioContext* 中首个新计算 PCM 帧被渲染，缓存音频续播不属于新计算。主指标为

$$T_{\text{first-new-PCM}} = t_{\text{new}} - t_{\text{W, resume}}$$

Save 使用安全暂停到资源就绪的区间，Ready 使用 RESUME 接受到执行发布的区间，二者均采用服务端单调时钟；Gap 使用浏览器时钟定义为 $\max(0, t_{\text{new}} - t_{\text{buffered-end}})$ 。这些端点不是串行片段，不能相加。

1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079

表 4: 状态覆盖与恢复义务。外部记录持续保留, 不被旧检查点副本覆盖。

状态类别	内容与恢复义务
模型	有效 KV、非 KV 历史、处理器 / 控制状态、采样上下文和逻辑位置。将数值恢复至合法映射, 并重建活动绑定。
桥接	已产生但未消费的 token、部分缓冲、初始化 / flush 状态和待提交输出。保留内容与消费顺序。
声学	StreamingDecoder 与 Token2Wav 连续状态、Flow / HiFT 缓存、实际随机上下文和待提交 PCM。恢复兼容的执行上下文。
身份与进度	请求、generation、状态版本、cut、lease epoch、sequence、producer epoch 和各阶段水位。根据当前生命周期验证归属。
外部记录	已接受输入、取消决定及交付 / 渲染区间。保留当前事实, 拒绝过期提交。

配置统计先在同来源、同环境内汇总具有对应端点的重复, 再取来源间中位数。配对差先匹配来源、重复和环境; 缺失值不以零替代。冷恢复、传输、吞吐量和交互批次分别保留身份, 不合并统计。

最终矩阵包含 96 条计划运行, 其中 92 条通过技术合同。Rebuild 保留 1 条 E1 暂停后原生浏览器帧重叠取消; Hybrid 保留 3 条 E1 失败 (两条帧重叠取消和一条冻结切面/首个 PCM 观测轮次不匹配)。Hot 和 Joint 没有失败, E2 全部通过技术合同。Hot 和 Rebuild 没有对应的检查点、pinned 缓冲或 GPU 工作区测量事件, 主表分别标记为 N/A 或 NM, 不填零。system pinned、 L_{useful} 和 $J(T)$ 不属于本轮冷恢复合同, 明确标记为未评价。

B 联合状态与恢复验证

表4总结联合恢复边界。计算状态可以恢复, 而已接受输入、取消决定与交付记录保持当前值。这一区分在请求粒度上应用一致快照与回滚恢复原则 (Chandy and Lamport, 1985; Elnozahy et al., 2002)。

捕获阶段先建立一致切面与独立主机检查点, 再交还请求私有资源。恢复阶段准备合法映射、重建绑定并验证各参与阶段状态, 之后才允许执行和输出。发布时重新检查当前 generation、版本和取消状态; 失败或已取消的恢复不被发布。恢复已保存的输入消费位置, 不会覆盖检查点之后收到的输入。

请求身份回归区分物理输出顺序 $[A, B]$ 与仅包含 B 的采样选择。request ID 查找取得 B 对应的 side output, 而不是物理第零行的输出。独立归档检查还覆盖桥接待处理内容、发布前拒绝和请求局部取消。完整 token 与波形比较以匹配的输入 / 控制历史及数值配置为条件, 不代表语义正确, 也不保证任意 kernel 的无条件确定性。

协议证明义务。 协议信封 q 包含五个字段: $request_id$ 、 $generation$ 、 $state_version$ 、 cut_id 和 $lease_epoch$; 每条记录再附带 $sequence$ 和 $producer_epoch$ 。证明采用如下归纳。初始状态只有一个归属者, 已产生集和已消费集均为空。SAVE 在切面线性化

表 5: 补充冷恢复测量, 单位秒。数值为对应环境内端点的来源级中位数。N/A 表示事件不适用, NM 表示没有对应测量; 二者都不是零。第一组给出最新报告中完整 96 次运行分母下的策略级有效率; 失败运行仍保留在分母中。

环境	策略	有效/总数	保存	就绪	首个新 PCM 渲染	间隔
所有环境 (最新 96 次运行批次)						
All	Hot	96/96	—	—	—	—
All	Rebuild	92/96	—	—	—	—
All	Joint	96/96	—	—	—	—
All	Hybrid	96/96	—	—	—	—
E1	Hot	11/11	0.004	0.001	1.184	0.349
E1	Rebuild	10/11	1.847	13.317	14.451	13.616
E1	Joint	11/11	1.387	0.699	1.899	1.064
E1	Hybrid	8/11	0.655	0.130	1.349	0.515
E2	Hot	13/13	0.004	0.001	1.168	0.333
E2	Rebuild	13/13	1.926	13.250	14.400	13.565
E2	Joint	13/13	1.387	0.700	1.872	1.037
E2	Hybrid	13/13	0.696	0.136	1.315	0.480

点前关闭全部写入者, 因此检查点内记录具有相同 q ; 排空在途提交后, 检查点与队列恰好划分已产生未消费集合。RESUME 只有在读取当前账本水位后才安装新的 lease epoch。之后每个提交只有在完整信封等于当前 permit 时接受, 账本更新与队列位置推进原子完成。

因此, 请求 ID 不匹配的记录会被拒绝, 归属保持不变; token 要么在检查点中, 要么由消费者事务恰好移除一次, 桥接守恒保持; 输入、取消和交付水位只增不减, 账本单调性保持。generation、state version 与 lease epoch 共同拒绝迟到 worker 及 ABA lease 复用。消费者只接受下一个 sequence, 故 PCM 前缀 / 顺序由归纳得到。PUBLISH 是离开 RESTORING 的唯一转换, 且只在这些检查完成后开放, 因此无效状态不能执行或可见。任一前提失败都转入 ABORTED, 不会发布半恢复状态, 也不会回滚外部事实。

两个线性化点在 trace 中可观测: CUT 是原子关闭并快照的事件, RESTORE 是原子安装新 lease 和 permit 的事件。取消在线性化时写入账本; 恢复若观察到该记录则中止, 而更晚的取消会在下一次提交前使 permit 失效。上述规则分别覆盖切面不一致、版本过期、ABA lease、迟到 worker、桥接重复消费、PCM 乱序、取消 / 恢复竞争及检查点后的新输入, 不依赖物理 worker 身份。

C 补充性能结果

表5补充正文的测量端点。表中同时报告保存、就绪、首个新 PCM 渲染和间隔, 每项均保留自己的起止锚点。各指标保留自己的起止端点, 不与其他区间相加构造新的端到端指标。

1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187

算法 1 逻辑请求 r 的保存 / 恢复协议。

输入: 当前信封 q 、状态 (M, B, A) 和外部账本 L

保证: 发布 permit, 或进入不可执行的中止状态

- 1: 原子关闭输入、token 生产和输出发布入口
 - 2: 递增 $state_version$ 并分配 cut_id
 - 3: 在支持的边界暂停 M, B, A 并排空在途提交
 - 4: **若** 存在信封不同于 q 的记录 **则**
 - 5: 拒绝该记录并中止捕获
 - 6: **结束 若**
 - 7: 在 CUT 线性化点执行 $K \leftarrow \text{SNAPSHOT}(M, B, A, L, q)$
 - 8: **若** $\neg \text{CHECKJOINT}(K)$ **则**
 - 9: 保留原资源; **返回** ABORTED
 - 10: **结束 若**
 - 11: 释放私有资源并将 r 标为挂起
 - 12: 读取当前 L ; 若取消或低于输入 / 取消 / 交付水位则拒绝
 - 13: 分配新的 $lease_epoch$ 并合并切面后的输入
 - 14: 从 K 重绑定 M, B, A
 - 15: **若** $\neg \text{CHECKJOINT}(M, B, A, L, q)$ **则**
 - 16: 丢弃未发布状态; **返回** ABORTED
 - 17: **结束 若**
 - 18: 在 RESTORE 点原子安装新 lease 和 permit
 - 19: **当** r 仍在执行 **执行**
 - 20: **若** 提交信封不等于 q , 或 PCM 序列不是下一个前缀值 **则**
 - 21: 丢弃该提交
 - 22: **否则**
 - 23: 原子追加账本记录并推进消费位置
 - 24: **结束 若**
 - 25: **结束 当**
 - 26: **返回** PUBLISHED
-

最终批次的首个新 PCM 渲染区间比缓存后供给间隔多 $20,032/24,000 = 0.834667$ 秒。这是预定义缓存播放时长，不是额外的时延项；二者是同一交付时间线的不同视角，不是独立改善。E2 中，Joint 与 Hybrid 的独立检查点载荷分别为 672.207 和 22.416 MiB，受管理 pinned 缓冲分别为 256 和 128 MiB。两者均保留模型设备工作区，使模型 GPU 净分配增加 60.489 MiB。检查点大小、pinned 存储、进程 RSS 和设备分配属于不同测量，不能直接相加。

独立传输实验包含六个来源组、12 对 Sync/Optimized 运行。每对的相对降幅为 $(T_i^{\text{sync}} - T_i^{\text{opt}})/T_i^{\text{sync}}$ 。其中传输到接收端就绪区间的相对降幅中位数为 16.00%，绝对减少量中位数为 0.533 秒，12 对均为优化路径更快。该批次没有测量 Worklet 恢复、首个新 PCM 渲染、播放结束或端到端恢复时延。额外暂停 RSS 与采样峰值 RSS 的配对增加中位数分别为 243.2 和 182.2 MiB。历史吞吐矩阵则对各负载 / 并发单元的逐次配对比值取均值。其全部配置和重复应完整保留；新吞吐测量作为独立批次处理。

D 交互与外部协议

自然交互先导采用 FD-Bench 风格的适配定义 (Peng et al., 2025)。SRR、SIR 与 SRIR 的分母分别保留符合条件的非打断问句、实际打断机会和成功打断事件。EIR 记录助手过早进入用户话轮。FSED 从用户语音结束计量至匹配回复开始播放，IRD 从用户插话开始计量至助手上一段语音结束。时延保留正负符号。显式 CPU 挂起不计作语音打断，拼接 PCM 也不能替代真实渲染时间线。

导出的事件列按如下方式计算 $\text{SRR} = k_{\text{SR}}/n_{\text{SR}}$ 、 $\text{SIR} = k_{\text{SI}}/n_{\text{SI}}$ 、 $\text{SRIR} = k_{\text{SRI}}/n_{\text{SRI}}$ 和 $\text{EIR} = k_{\text{EI}}/n_{\text{EI}}$ ；某指标列缺失时，该事件不进入该指标分母。每条受控路径有 10 个有效问句、3 个实际打断机会，6 次重复中无效 / 取消运行数为 0。唯一非零的失败相关字段是 Omni 的 1 次 reply timeout，该记录不产生 FSED 观测，因此该路径 FSED 有效 $n = 9$ 。正文区间是 source-level bootstrap 区间 (对 6 个 source group 重采样 200,000 次，seed 为 20260926)；有效 bootstrap 次数保留在 `interaction_metrics_bootstrap.csv` 中。

共享 Lychee/DuplexPilot 在线路径属于同一实现，不是两个独立基线。该路径采用 400 ms 块末端输入，Omni 客户端采用 200 ms 先发送、再等待的输入方式；模型和实际打断暴露状态也不同。因此，先导描述应用行为，不用于隔离冷恢复影响。此前听音反馈只适用于已经审阅的案例，不扩展到未审阅先导输出。

两例 vLLM-Omni 原生热重附着 (vLLM-Omni Contributors, 2026) 的请求至 ACK 分别为 1.512 和 1.421 ms，首 PCM 接收分别为 725.412 和 776.220 ms。两例记录中的 PCM 序号没有缺失或重复，但首个返回 PCM 的产生阶段未知。LiveServe 风格预加载组件 (Zhi et al., 2026) 提前约 9.9 秒准备 20 和 15 MiB KV。其就绪分别提前约 56.65 和 100.01 ms，但从共同源窗口计量的首新 PCM 接收分别晚 23.7 和 36.4 ms。这些观察保留各自原生协议，不构成统一的外部冷恢复比较。

1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295

表 6: 受控 Lychee/DuplexPilot 路径和原生 Omni 参照路径的逐 source 重复结果。比例为分子/分母；FSED 和 IRD 为带符号播放层中位数 (ms)，括号内为有效 n 。每次重复均未出现无效或取消运行；反向通道排除是有意的非指标事件，不是失败。

配置	source group	SRR	SIR	SRIR	EIR	FSED (ms)	IRD (ms)	timeout	无效/取消
Lychee	0001_0001	1/1	–	–	0/1	3666.4 (1)	–	0	0
Lychee	0001_0003	1/1	–	–	0/1	3762.9 (1)	–	0	0
Lychee	0001_0015	1/1	1/1	1/1	1/2	5582.8 (2)	266.6 (1)	0	0
Lychee	0001_0025	1/1	1/1	1/1	1/2	5575.1 (2)	278.2 (1)	0	0
Lychee	0001_0029	2/2	–	–	1/2	5808.0 (2)	–	0	0
Lychee	0001_0031	1/1	1/1	1/1	1/2	3615.5 (2)	898.9 (1)	0	0
Omni	0001_0001	1/1	–	–	1/1	-185.9 (1)	–	0	0
Omni	0001_0003	1/1	–	–	1/1	1324.7 (1)	–	0	0
Omni	0001_0015	1/2	–	–	2/2	302.9 (1)	–	1	0
Omni	0001_0025	1/1	1/1	1/1	1/2	1507.4 (2)	3444.6 (1)	0	0
Omni	0001_0029	1/1	1/1	1/1	2/2	1351.6 (2)	564.1 (1)	0	0
Omni	0001_0031	1/1	1/1	1/1	2/2	1544.5 (2)	2068.3 (1)	0	0